

我最近参加了七牛 1024 实训营，做的是一个 code agent 项目：[1024XEngineer/neo-code](#)。

这篇不是想泛泛介绍多智能体架构。相反，是一次项目复盘：我一开始确实想把 SubAgent、Todo、Scheduler、DAG 都做起来，但后来在会议讨论和实际排障里发现，当前阶段最重要的不是先把并发编排做复杂，而是先把**单 Agent 主链路闭环和可控的 inline 子代理**做稳。

许总和飞龙老师在会议里反复提醒的一点，对我影响很大：架构设计要从业务问题出发，不要先从实现细节出发。对一个 AI 编程助手来说，真正核心的问题不是“能不能多开几个模型”，而是：

- 用户的问题有没有被拆清楚；
- 工具有没有产生可验证事实；
- 任务完成是不是有工程判定，而不是模型自己说完成；
- 单会话闭环是否稳定；
- 团队能不能在里程碑前交付、验证、维护。

为什么需要 SubAgent

用户经常给出这种任务：

帮我修一个复杂 bug，顺便解释原因，改完后跑测试，再检查有没有引入风险。

如果把这些事情全部塞在一个主代理 ReAct 循环里，问题会很快出现：

- 上下文污染：无关工具输出不断堆积；
- 工具结果过多：主循环被日志、中间结果、失败尝试淹没；
- 模型偏题：做着做着忘掉最初目标；
- 验收标准丢失：模型容易“自我宣布完成”；
- 任务分工不清：诊断、修复、验证、review 混在一起；
- 用户不可观测：用户看不清到底哪个子任务完成了。

所以我给 SubAgent 的定义是：

SubAgent 的价值不是“多开一个模型”，而是把复杂任务中的某个子问题切出来，用受控上下文、受控权限、受控输出完成，再把结果回灌给主代理。

这里必须先问一个团队开发里很重要的问题：**我能不能不做？**

答案是：短任务可以不做。单 Agent 已经足够。比如简单问答、读一个文件、改一个小 bug，都不一定需要 SubAgent。

但复杂任务里，review、verify、diagnosis、limited edit 这类子任务会反复污染主上下文。主代理既要读代码，又要判断风险，又要验证，又要组织最终回答，链路越长越容易漂移。SubAgent 的产品价值就在这里：它把局部任务隔离出来，让主代理保留全局决策，子代理只做受限任务。

ReAct、SubAgent、inline、DAG、Coordinator

讨论 SubAgent 时，很容易把几个概念混在一起。我现在会这样拆：

概念	解决什么问题	在 NeoCode 当前阶段的状态
ReAct	单个代理如何思考、调用工具、观察结果、继续行动	主代理和子代理都可以用

概念	解决什么问题	在 NeoCode 当前阶段的状态
SubAgent	主代理之外承接单一子任务的受控执行单元	当前做最小闭环
inline	主代理显式调用一个子代理，并等待结果回灌	当前采用
bounded queue	多个子任务排队，有限并发执行	下一阶段
DAG	带依赖关系的任务图编排	暂缓
Coordinator	中心化调度多个 worker 的编排角色	后期

一句话区分：

ReAct 是单个 Agent 的工作方式；inline 是主 Agent 调用子 Agent 的同步方式；DAG 是多个子任务之间的编排方式；Coordinator 是更高层的调度角色。

这几个不是同一层东西。把它们混在一起，就容易一上来做出一个“看起来很完整、实际很难收敛”的系统。

为什么选择 inline 最小实现

我最早在多代理方向上是想做 Task DAG 的。

这条路线看起来很自然：

- 复杂任务可以拆成多个节点；
- 节点之间可以有依赖；
- 不同 worker 可以并发；
- 失败后可以重试或恢复；
- 最后由 aggregator 合并结果。

从长期看，DAG 没有错。它确实是复杂任务编排的一个重要形态。

但问题在于：**它不适合在单会话闭环还没稳定时成为主路径。**

会议里有一个共识对我影响很大：当前阶段应该优先保证单会话闭环稳定，而不是先追求并发。因为 coding agent 的基础能力不是“先接多少模型”或“先开多少 worker”，而是读、写、改、测、Git、判定工具这些基础链路是否闭合。

我后来意识到，DAG 主路径太早会带来几个风险：

- Todo 状态、工具状态、权限状态、验收状态会同时演化；
- 排障时很难判断是调度错、执行错、回灌错还是判定错；
- worker、scheduler、result merge、retry、cancel 会一起变复杂；
- 没有 Git / workspace change tracking / 验收事实支撑时，多 worker 反而会互相污染；
- 对团队来说，issue 范围过大，里程碑前很难证明“稳定且可维护”。

这不是“代码写不出来”的问题，而是**无法在截止前给出可信交付**的问题。

所以我把目标收敛成：

先保证单 Agent 主链路能闭环；
Todo 只表达任务状态，不表达自动调度；
需要子任务时，显式调用 `spawn_subagent(mode=inline)`；
子代理只拿任务切片、能力边界和输出契约；
执行结束后，以结构化 `ToolResult` 回灌主代理。

这不是技术降级，而是控制变量。

工程优先级

第一，架构设计要从业务问题出发，而不是从实现细节出发。

我以前容易先想：SubAgent 怎么调度？DAG 怎么建？Scheduler 怎么写？后来发现，应该先问：用户在哪场景下需要它？不做会怎样？做了能解决什么问题？

第二，Agent Loop 才是系统核心。

Provider 很重要，但它主要是模型适配层。真正决定 code agent 能力上限的是 Agent Loop：任务拆解、工具协作、反馈修正、完成判定。SubAgent 也必须服务这个闭环，而不是独立炫技。

第三，工具链完备性优先于复杂能力堆叠。

如果 read/write/edit、bash、Git、测试、判定类工具都没稳，SubAgent 再复杂也只是调度一个不稳定系统。

第四，单会话闭环稳定优先于复杂并发。

这直接决定了我把 DAG 暂缓，把 inline 放到当前阶段。

所以我现在写 issue 或做模块时，会先回答这两个问题：

1. 这个功能对用户增加了什么价值？
2. 这个 issue 在里程碑前的风险、卡点、返工概率是什么？

如果这两个问题答不清楚，就不应该贸然把范围做大。

SubAgent 给用户带来什么

场景 A：复杂代码 Review

用户说：

帮我 review 这个 PR 有没有安全风险。

如果不做 SubAgent，主代理要同时读代码、归纳逻辑、判断风险、组织结论。上下文一长，就容易漏掉局部风险，review 过程也不可独立观测。

做了 SubAgent 后，主代理可以派一个 reviewer 子任务：

- 子代理只拿相关文件和 review SOP；
- 输出 findings / risk / evidence；
- 主代理负责最终汇总和取舍。

用户得到的不是“另一个模型说了很多话”，而是一个更可消费的 review 结果。

场景 B：修复后的验证

用户说：

修完这个 bug 后帮我确认测试有没有过。

如果不做 SubAgent，主代理容易把“我觉得修好了”当完成。修改过程和验证事实混在一起，最后验收容易变成自然语言自证。

做了 SubAgent 后，可以把 verify 切成一个受控任务：

- 子代理专注跑测试或检查验证事实；
- 输出结构化验证结果；
- 主代理基于事实做最终答复。

场景 C：长任务降噪

复杂任务里，主代理会读很多文件、跑很多命令、产生很多中间日志。如果这些都直接堆进主上下文，主代理会越来越慢、越来越容易跑偏。

SubAgent 的价值是把局部探索隔离开：子代理只回灌摘要、发现、产物和状态，不把完整过程倒回主上下文。

总结一句：

SubAgent 的产品价值不是提高并发数量，而是降低复杂任务的认知负担：让局部任务可隔离、可观测、可审计、可回灌。

spawn_subagent(mode=inline)

NeoCode 当前采用的是显式 inline 子代理，不是 Todo 自动调度。

链路是：

```
Main Agent decides subtask
-> calls spawn_subagent(mode=inline)
-> tool schema validation
-> build SubAgentRunInput
-> runtime-injected SubAgentInvoker
-> capability narrowing
-> RunSubAgentTask
-> SubAgentRunResult
-> structured ToolResult
-> main loop continues
```

每一层职责都要清楚：

- 工具层只做协议和参数校验；
- runtime 负责真正执行子任务；
- capability narrowing 保证权限只收敛不放大；
- result 回灌为结构化对象，不是聊天转储；
- 主代理仍然是最终决策者。

我特别强调“显式调用”，是因为之前踩过一个坑：Todo 里出现过 `executor=subagent` 这类语义，模型会误以为写了这个字段就能自动派发子代理。实际 runtime 没有这个自动调度链路，结果就会出现模型反复说“我会派发 subagent”，但没有真实工具调用。

这不是技术降级，而是控制变量。当前阶段的目标不是做出最完整的多 Agent 编排形态，而是先证明单 Agent 主链路、工具调用、权限边界和结果回灌能稳定闭环。

最后我们把这个语义收掉：**Todo 只表示任务状态，spawn_subagent 才是子代理显式入口。**

安全边界

```
child_capability = parent_capability ∩ child_requested_capability
```

- `allowed_tools` 只能收敛；
- `allowed_paths` 只能收敛；
- 子代理不能拿到父代理没有的工具；
- 子代理不能访问父代理没有授权的路径；
- `prompt / content` 是任务文本，不是路径授权；
- 越权必须结构化失败，不能让模型猜。

用户授权主代理访问某些文件，不代表系统可以因为“拆了一个子任务”就扩大权限边界。任务拆解不能改变授权边界。

这条规则牺牲了一点灵活性，但换来一个很重要的性质：系统权限边界是可解释、可审计、可证明的。

子代理怎么回灌才不污染主循环

我现在把防污染拆成四层。

第一，输入限幅。

`prompt`、`content`、`allowed_tools`、`allowed_paths` 都要限制长度和数量，防止模型把大段上下文偷渡给子代理。

第二，上下文切片。

子代理不继承主代理完整历史，只拿任务目标、必要依赖和验收条件。

第三，输出结构化。

回灌的是 `summary / findings / artifacts / status / error` 这类结果字段，而不是完整聊天记录。

第四，输出限长和错误分类。

`timeout`、`canceled`、`permission_denied`、`contract_violation`、`budget_exceeded` 要区分。大输出要截断或摘要，避免主上下文爆炸。

子代理不是第二个聊天窗口，而是一个受控任务函数。主代理拿到的是结果对象，不是过程噪声。

落地过程中的关键修正

Todo executor 字段误导模型

现象：

- 模型写了 `executor=subagent` 就以为会自动派发；
- 实际 runtime 没有自动调度；
- 模型一直“口头派发”，没有真实工具调用；
- 用户看到的是重复话术，而不是实际执行。

修复：

- 移除 Todo 协议里的 executor 语义；
- Todo 只表达任务状态；
- `spawn_subagent` 成为唯一显式入口。

DAG 主路径太早

现象：

- scheduler、worker、todo、permission、verification 一起复杂化；
- 排障时不知道是调度错、状态错、工具错还是验收错；
- issue 看起来很完整，但截止前无法证明稳定。

修复：

- 暂缓 DAG 主路径；
- 保留顺序主链；
- 先做 inline 最小闭环。

单循环没稳之前，并发通常只会放大不确定性。

权限边界容易被忽略

现象：

- 子代理如果独立配置 `allowed_tools` / `allowed_paths`，可能大于父代理；
- 这会变成权限逃逸。

修复：

- 子权限 = 父权限 $\cap$ 子请求权限；
- 超边界即拒绝；
- 路径必须做归一化和覆盖判断。

子代理权限不能高于父代理，这是安全底线，不是产品选项。

结果回灌不结构化会污染主循环

现象：

- 子代理输出太长；
- 主代理把子代理过程文本当事实；
- 后续推理被污染；
- 用户看不到结构化结论。

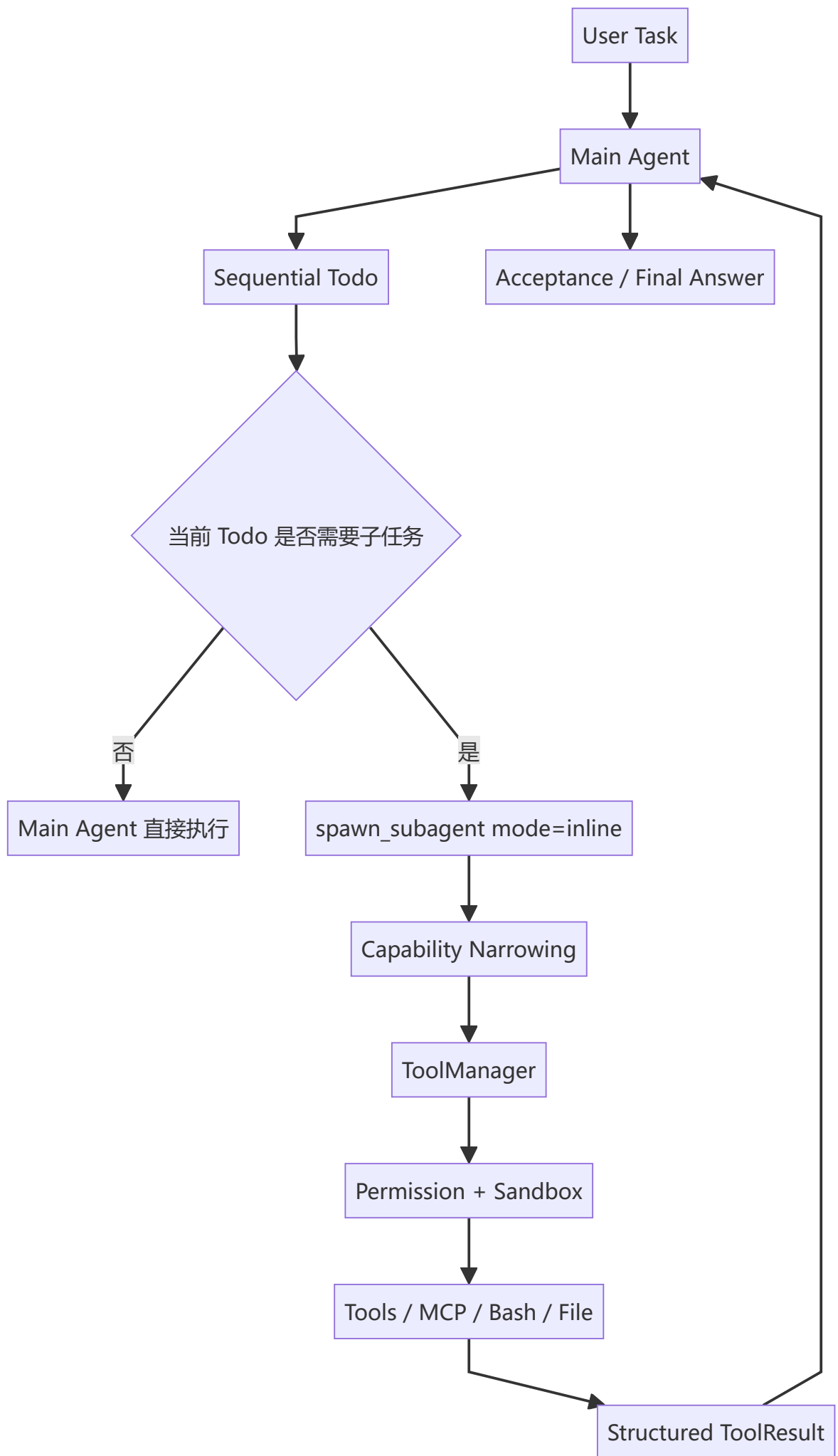
修复：

- 固定 ToolResult 结构；
- 限长；
- 错误分类；
- 只回灌 summary / findings / artifacts / status。

输出不是越多越好，而是越可消费越好。

当前进度

模块 / issue	用户价值	当前实现	截止前风险	是否可能返工	应对策略
spawn_subagent inline	主代理能隔离子任务，减少主上下文污染	已做最小闭环	输出契约不够强，模型可能滥用	中	固定结构化结果，补 prompt 和测试
capability narrowing	防止子代理越权，保护用户授权边界	已做收敛	路径归一化、Windows 大小写、相对路径绕过	中高	增加路径覆盖和跨平台测试
Todo executor 语义收敛	避免模型误解“写 Todo 就会调度”	已修	旧 prompt / 文档残留旧语义	低	同步更新 prompt、docs、测试
DAG / scheduler 主路径	支持复杂任务并发编排	暂缓	范围过大，截止前不可控	高	不进本轮主线，拆后续 issue
bounded queue	下一阶段可控并发	未做	cancel / timeout / retry / result merge 语义未闭环	中	等 inline 稳定后单独设计



核心约束：

1. Todo 只表达任务状态，不表达自动调度；
2. `spawn_subagent(mode=inline)` 是子代理唯一显式入口；
3. SubAgent 工具调用必须经过 ToolManager + Permission + Sandbox；
4. 子代理结果以结构化 ToolResult 回灌，而不是完整聊天记录；
5. 主 Agent 保留最终决策权。

以前我会更容易被架构形态吸引：DAG、Coordinator、多 worker、并发调度，这些都很像一个完整系统。但现在我会先问：

- 这个功能给用户增加什么价值？
- 不做它会有什么真实问题？
- 当前里程碑能不能证明它稳定？
- 如果失败，系统是否能解释？
- 团队接手时，边界是否足够清楚？
- 这个 issue 是否会拖累别人的模块？
- 是否需要依赖 Git、工具链、验收、TUI 等其它模块先稳定？

答案是：

先做好单 Agent 主链路；
再做好工具链和工程判定；
再做好 inline 子代理；
最后再讨论 bounded queue、DAG、Coordinator。

任务不是越复杂越好。先把 inline 做成可信执行单元，再谈 DAG 和 Coordinator，才是对用户、团队和项目进度都更稳的路线。

(完)

关于其他

[Hooks: 生命周期扩展点 | Ca1 Tang](#)

[NeoCode 架构分析 | Ca1 Tang](#)

[Human-in-the-Loop: 人不能退出架构决策 | Ca1 Tang](#)